SAGA Distributed Transactions Verification Using Maude

Manel Djerou
Computer science department
University of Biskra, Algeria
Biskra, Algeria
djeroumano@gmail.com

Okba Tibermacine
Computer science department
University of Biskra, Algeria
Biskra, Algeria
o.tibermacine@univ-biskra.dz

2022 International Symposium on iNnovative Informatics of Biskra (ISNIB) | 979-8-3503-2065-7/22/$31.00 ©2022 IEEE | DOI: 10.1109/ISNIB57382.2022.10076050

***Abstract*—Microservices is an architectural style that enables the design of applications as series of services independent from each other in function, maintainability, deployability and scalability. Two phase commit protocol as an example of classical distributed transaction protocols are not suitable for this pattern. These protocols require all participants in a transaction to commit or roll back before the transaction can proceed. This is way saga was chosen to replace 2PC. Saga is a mechanism that belongs to the 'database per service' pattern and which guarantees distributed transactions that covers many services. The saga pattern provides transaction management using a sequence of local transactions. Saga can be rolled back by a compensating transaction that ensures one of two possibilities all operations complete successfully or the corresponding compensation transactions are run to undo the change. In this paper, we try to run a verification of saga using the maude language. We use the Saga protocol by orchestration, and we present a verification way to check it.**

***Index Terms*—Maude Language, Microservice, Saga, Orchestrator, Verification, Service, FTGO application.**

I. Introduction

In our daily lives, software development has created significant advancement in numerous fields. Microservices enables for the creation of software applications through a collection of small, independent software units known as services, among those applications are: Amazon, Netflix, eBay, Paypal. This architectural styles fundamental objective is to ensure independent maintainability, deployability and scalability allowing applications to develop and grow more sophisticated without shifting the presence of traditional issues.

The two phase commit protocol and other traditional distributed transaction protocols are not a good fit for this structure. This is because these protocols demand that before a transaction can be completed, all parties involved in it must either commit or roll back. Alternatively, saga is a productive mechanism that adheres to the "database per service" architecture and guarantees distributed transactions that span numerous services. Using a series of local transactions, the saga pattern offers transaction management. That last is the type of job a saga participant does. A compensating transaction, which ensures that either all operations succeed or the relevant compensation transactions are run to undo the work already done, can roll back any operation that is a part of the saga.

In this research, we outline a classy method for orchestrating the saga protocol. The executable specifications produced by the maude language and its verification capabilities are also used to demonstrate an effective technique to verify their accuracy.

Finally, the remaining parts of this paper are organized as follows. Section 2 presents related works. Section 3 presents basics necessary to understand language used and recent microservice architecture and the saga meaning and explain the application example used. Section 4 gives details about our program of saga specification using maude. Section 5 presents verification of our studies. Finally, section 6 concludes the paper.

II. Related Work

We can find in the literature some works about formal verification such as [1],[2] they are based on different formal languages such as mCRL2 where in this work [3] the obtained model is created using rewrite rules with the addition of process declaration usage. Which depends on multiple languages where in our work we utilize rewrite rules through maude to obtain a model. Other works are based on formal verification using maude language such as [4], [5], [6]. As far as we know, this is the first work obtained in this paper to formally verify saga transaction.

III. Basics

A. Maude

Maude is a formal specification and declarative programming language used to express the semantics of a maude system[7; 8]. Unlike imperative languages like C or Java. [9] Maude is a clear, expressive and powerful language [7] built on the mathematical theory of rewriting logic. [10]

The rewriting logic and the maude language were developed by Jose Meseguar and his team at SRI International's laboratory. [7] Rewriting logic is a dynamic logic that can handle real-time calculations. It is compatible with object-oriented computation. It's an adequate semantic and logical structure. [7; 8]

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 08:52:45 UTC from IEEE Xplore. Restrictions apply.

Maude is considered one of the top languages for algebraic specification and modeling of concurrent systems [7] Maude is now utilized in companies around the world for concurrent and distributed system training, research, formal modeling, and analysis. [9] [11] [12]

It is distinguished by its clarity, ease of expression, efficiency, and performance. The Maude project's objectives were to back executable formal specifications and to widen the use of declarative programming and formal methods for specifying, implementing, and analyzing best quality systems in fields like communication networks, bioinformatics, and distributed computing, among others. [13]

For specifying a distributed system, maude provides three modules: [7] Modules that are functional, system and object-oriented (used in our work). Data types and operations in functional modules are defined using signatures and equations similar to simplification rules. Rewrite rules introduced in system modules, so, determine a system's behavior. These rules have the form "R: [t] —+ [t'] if C," which signifies that if a condition C is satisfied, term t rewrites to t', and C is optional .

Finally, object-oriented modules enable for more accurate modeling of object configurations and messages. Maude suggests "full maude" for later consideration. [14]

B. Microservice and Saga

1) Microservice: The term " microservice " expresses a system made of different processes that operate independently from each other and function as part of a joint application.

The microservice architecture appeared recently as an alternative to classical application design architecture (Monolith) . It is based on the concept of creating an application by combining several services that run independently from each other and inter-communicate. [15; 16]

Microservices communicate with each other using distributed transactions.

Classical distributed transactions (two phase commit protocol) are not compatible with the micro service architecture because the 2PC's pattern requires all participants to either commit or rollback which is not practical for microservice systems.

To avoid these problems we opted for Saga as a solution.

2) Saga: Saga is simply defined as a series of local transactions that update multiple services in a microservices system.[17]

There are two ways of saga :

- **Choreography based saga.**
- **Orchestration based saga.**

Orchestration based saga was chosen because of its simplicity to implement and its ease of use.

Saga is able to manage multiple transactions originating from several microservices using local transaction sequences. [17]

C. Illustrative Example

At this point, we will explain a real example of an existing application which we used this for the verification of distributed transaction saga. In particular, we aim to verify one of ways of saga with maude language. This way is the saga orchestration pattern.

The application we used is Food To Go or briefly FTGO.

1) FTGO Application Description: FTGO is a distributed applications that facilitate the ordering, ticketing, payment and the delivery of food orders requested by online users using their means of communication. The main goal of the program is networking between consumers, local restaurants and couriers. When a customer places an order with a local restaurant using the FTGO application, the request is coordinated linking the restaurant and the couriers, followed by the organization of payment to the restaurant and the couriers. [17]

IV. Saga Specification Using Maude

To specify saga using maude language. Firstly, we divided our work into five parts to represent a solution and try to reduce the number of problems by using the verification of some properties in the saga to guarantee a better result.

Now, we have five modules in our source code using maude.

The first module is **MESSAGE-CONTENT** , the second is **MESSAGE-WRAPPER**, the third is **MESSAGE-BROKER**, the fourth is **SERVICE-DEFINITION** and the last is **INITIAL-CONFIGURATION.**

We start to define the modules. For **MESSAGE-CONTENT** module has just a sort **"MsgContent"** .

We have defined **MESSAGE-CONTENT** by Maude in this form :

```
(mod MESSAGE-CONTENT is
sort MsgContent . --- message content, application-
    specific
endm)
```

For second module, we are declaring forms of message by operations. This operations assist the implementation of our solution and we are including the first module for using its sort.

We have defined **MESSAGE-WRAPPER** by Maude in this form :

```
--- Message wrapper
(omod MESSAGE-WRAPPER is including MESSAGE-CONTENT .
op msg_from_to_ : MsgContent Oid Oid -> Msg .
op msg_to_ : MsgContent Oid -> Msg .
op msg_from_ : MsgContent Oid -> Msg .
endom)
```

In the **MESSAGE-BROKER** module, we are modeling the links of communication between services by creating class **MBroker** where the only attribute in this class is **MsgContentList** and included the first module.

And we have represented this module by Maude in this form:

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 08:52:45 UTC from IEEE Xplore. Restrictions apply.

```
--- Module Broker( Links)
(omod MESSAGE-BROKER is including MESSAGE-CONTENT .
sorts MsgContentList .
subsort MsgContent < MsgContentList .
subsort Msg < MsgContentList .
op nil : -> MsgContentList [ctor] .
op _::_ : MsgContentList MsgContentList ->
    MsgContentList [ctor assoc id: nil] .
op _to_ : Oid Oid -> Oid [ctor] . --- link names
op allto_ : Oid -> Oid [ctor] .
class MBroker | content : MsgContentList .
endom)
```

In the **SERVICE-DEFINITION** module, we needed both other modules such as **MESSAGE-WRAPPER**, and **MESSAGE-BROKER** for including. We defined the declaration of module by maude in this form:

```
(omod SERVICE-DEFINITION is
including MESSAGE-WRAPPER .
including MESSAGE-BROKER .

--- OidSet definition : pour definir un ensemble d'
    objets
sort OidSet . subsort Oid < OidSet .
op none : -> OidSet [ctor] .
op _;_ : OidSet OidSet -> OidSet [ctor assoc comm id
    : none] .

--- Message List declaration
sort MsgList .
subsort Msg < MsgList .
op noMsg : -> MsgList [ctor] .
op _//_ : MsgList MsgList -> MsgList [ctor assoc id:
     noMsg] .
sorts State ComponstionMode .
```

This module is composed of class **SERVICE**, this class is formed of attributes, values, operations, sorts and messages.

We have defined the class service with its attributes **(isOrchestrator, approved, otherSERVICE, state, mode, OrchestrationStack, RollbackStack, recievedMsgs, MessageToSend and anyServFailed)** and sorts **(Bool, OidSet, State, ComponstionMode, MsgList and Msg)** where the definition in the model is:

```
--- Definition of service Class that represents a
    microservice
class SERVICE | isOrchestrator : Bool, approved :
    Bool, otherSERVICE : OidSet, state : State,
mode : ComponstionMode, OrchestrationStack : MsgList
    , RollbackStack : MsgList,   recievedMsgs :
    MsgList, anyServFailed : Bool .
```

We have defined the meaning of service class attributes in short as :

- **approved : "main attribute"**, true if database components are approved.
- **isOrchestrator :** true if a service is orchestrator.
- **otherSERVICE :** all other services in the system.
- **state :** state in saga rchestrator.
- **mode:** functions as normal automatically, but changes to abnormal if any service fails.
- **OrchestrationStack :** this attribute handles all the requests sent by the orchestrator.
- **RollbackStack :** the initial operations is null and it is specified to orchestrator saga, add message rollback if the service responds by **Ack** message, and use the messages adding if any service fails. The orchestrator sends a message rollback to services and responded with ack to reject any operation .
- **recievedMsgs :** any messages the service received.
- **MessageToSend :** all the reply messages that any service other than orchestrator can send for response to orchestrator saga.
- **anyServFailed :** true (in the orchestrator) if any service failed.

And we have defined all attributes of service class by the operators (sorts) and messages that the **SERVICE** class needs and uses by maude in this forme:

```
---All the operators and messages needs and uses
    SERVICE class.
ops initial sended ready willRespond available done
    failed rollbacked : -> State [ctor] .
ops waiting : Oid -> State .
ops VerifyConsumer CreateTicket AuthorizeCard
    ApproveTicket ApproveOrder RejectOrder : ->
    MsgContent [ctor] . ---All the messages the
    Orchestrator can be sending to another service.
msgs ack noAck rollback : Msg -> Msg .   ---
    Answering Messages
ops normal  abnormal : -> ComponstionMode [ctor] .
    ---Mode
ops ConsumerServ KitchenServ AccountingServ
    OrderServ : -> Oid [ctor] . ---Services Name
op orchSaga : -> Oid [ctor] .  ---Orchestrator Name

msg CreateOrder : Oid -> Msg .
```

In the fourth module, we have a part for message modeling. This part is specified as messages represents how sequence events happen into services. This message is represented by rewriting logic rules.

We are modeling many rules to allow the implementing of some proprieties for verify the saga.

Each rule represents working method and each rule has label containing specified name.

We put one of rules of rewriting logic like an example represented by Maude in this form:

```
rl [ Sending-Message] :
< O : SERVICE | isOrchestrator : true, approved :
    false, otherSERVICE : OS , state : ready , mode
    : normal ,
OrchestrationStack : (msg MC to O') // ML ,
    RollbackStack : noMsg , anyServFailed : false >
< O to O' : MBroker | content : MCL >
=>
< O to O' : MBroker | content : MCL :: (msg MC to O
    ') >
```

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 08:52:45 UTC from IEEE Xplore. Restrictions apply.

```
< O : SERVICE | isOrchestrator : true, approved :
    false, otherSERVICE : OS , state : waiting(O') ,
mode : normal , OrchestrationStack : ML,
    RollbackStack : noMsg, anyServFailed : false > .
```

In the first rewriting rule we have showed how the orchestrator started and changed your state from initial to ready .

The rest of the rules of rewriting logic by Maude can be found in the full module which uploaded through a Google Drive link in [18].

For the last module, the configuration is represented like an example allowing the testing of our programs.

In this part exactly, we used and tested rewriting rules .Also, in this part of configuration we are starting with part initialization of declaration of a services in initial state .

We have four services defining (" ConsumerServ ; KitchenServ ; AccountingServ ; OrderServ") and service orchestrating names orchSaga.

For linking between any service and saga orchsetrator, we used MBroker class.

We have defined this module by maude in this form:

```
op initState : -> Configuration .  ---this is
    operator represents a Configuration.

eq initState = CreateOrder(orchSaga) < orchSaga :
    SERVICE | isOrchestrator : true, approved :
    false, otherSERVICE : ( ConsumerServ ;
    KitchenServ ; AccountingServ ; OrderServ ),
    state : initial, mode : normal,
    OrchestrationStack : (msg (VerifyConsumer) to (
    ConsumerServ))//(msg (CreateTicket) to (
    KitchenServ))//(msg (AuthorizeCard) to (
    AccountingServ))//(msg (ApproveTicket) to (
    KitchenServ))//(msg (ApproveOrder) to (OrderServ
    )), RollbackStack : noMsg, recievedMsgs : noMsg,
     anyServFailed : false >
--- Consumer Service
< ConsumerServ : SERVICE | isOrchestrator : false,
    approved : false, otherSERVICE : ( orchSaga ;
    KitchenServ ; AccountingServ ; OrderServ ),
    state : initial, mode : normal,
    OrchestrationStack : noMsg , RollbackStack :
    noMsg, recievedMsgs : noMsg, anyServFailed :
    false >
--- Kitechen Service
< KitchenServ : SERVICE | isOrchestrator : false,
    approved : false, otherSERVICE : ( orchSaga ;
    ConsumerServ ; AccountingServ ; OrderServ ),
    state : initial, mode : normal,
    OrchestrationStack : noMsg , RollbackStack :
    noMsg, recievedMsgs : noMsg, anyServFailed :
    false >
--- Accounting Service
< AccountingServ : SERVICE | isOrchestrator : false,
     approved : false,
otherSERVICE : ( orchSaga ; KitchenServ ;
    ConsumerServ ; OrderServ ), state : initial,
    mode : normal, OrchestrationStack : noMsg ,
    RollbackStack : noMsg, recievedMsgs : noMsg,
    anyServFailed : false >
--- Order Service
< OrderServ : SERVICE | isOrchestrator : false,
    approved : false, otherSERVICE : ( orchSaga ;
    ConsumerServ ; AccountingServ ; KitchenServ ),
    state : initial, mode : normal,
    OrchestrationStack : noMsg , RollbackStack :
    noMsg, recievedMsgs : noMsg, anyServFailed :
    false >
--- brokers
< ((orchSaga) to (ConsumerServ)) : MBroker | content
     : nil >   --- broker between Orchestrator and
    Consumer Service
< ((orchSaga) to (KitchenServ)) : MBroker | content
    : nil >    --- broker between Orchestrator and
    Kitchen Service
< ((orchSaga) to (AccountingServ)) : MBroker |
    content : nil >  --- broker between Orchestrator
     and Accounting Service
< ( allto (orchSaga)) : MBroker | content : nil > .
                --- broker from any service to the
    Orechestrator
endom)
```

The full model can be found uploaded through a google drive link in [18].

V. VERIFICATION

In our work, we focus on the verification of saga using maude system. To verify the saga, we need the execution commands of maude.

In this part, we execute commands line in the terminal or we write all commands in the same file of program after that we use other commands line in the terminal for reading and calling the file after the execution. Finally, we get the results.

In general, to use maude language, we have methods and command-lines specified to this language. Firstly, you must open text editor and write all your program and save the file by name containing **.maude** for example : "**saga.maude**" .

To verify the file of maude from errors and confirm the solution, you must execute the program and search by commands lines for bugs. You have a choice for how to do execution commands if in the terminal or write in the program and call it in the terminal.

After the execution, if the results syntax's errors free, this is good but also it must be semantics's errors free. This is meaning a correct solution.

The maude system is fully operational and ready to take orders or modules.

During a session, the user interacts with the system by typing a request into the "maude prompt." For example, if one desires to quit maude:

$$Maude > quit$$

All full-maude modules must be included in parentheses if executed them.

(rew ....)

(search ....)

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 08:52:45 UTC from IEEE Xplore. Restrictions apply.

We can begin a session with maude on UBUNTU 20.04 by clicking on the terminal after opening.

We run the maude command to display the main maude window.

1. For executing this model , we used **"in saga.maude "**. This command line is used for reading the file in maude and verify the syntax of file . Result can be displayed in figure 1.

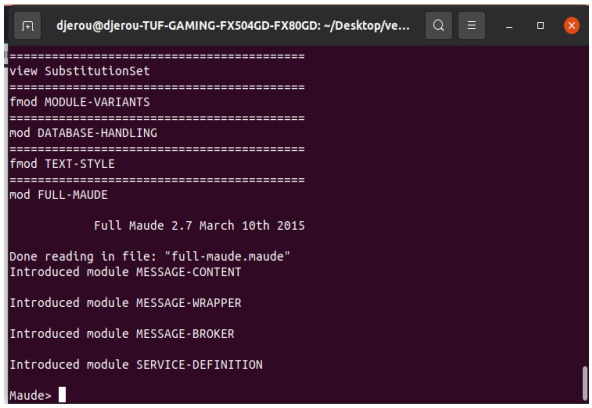


Fig. 1. Result of the execution in command

2. After that, we used **(frew [100] initState .)** command line in the terminal to try the execution of the model. Result can be displayed in figure 2.

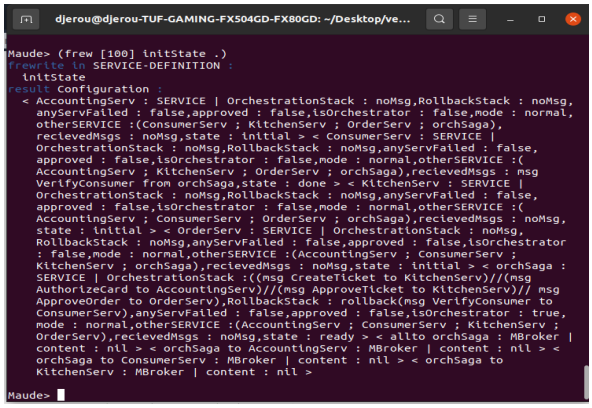


Fig. 2. Result of the execution frew command

3. Used this command line in the terminal search for checking the file in many side with many ways . In the first example of the execution we used this command search this way for checking if we can arrive to a compensation case from the initial one . Result can be displayed in figure 3.

```
( search [1] initState = >! < O : Oid : SERVICE |
    isOrchestrator : true , anyServFailed : true > C
     : Configuration .)  --- search a fault
```

And in the second example of the execution we used search command with another way for check if service orchestrate change to state ready . Result can be displayed in figure 4.

```
( search [1] initState = >! < O : Oid : SERVICE |
    isOrchestrator : true , state : ready > C :
    Configuration .)  --- search if service orch
    change to state ready
```

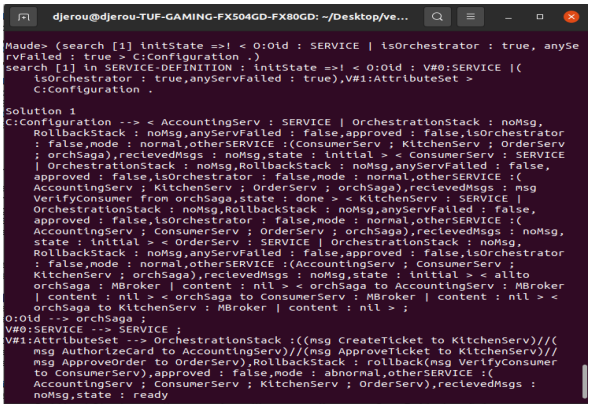


Fig. 3. Result of the execution search command for check if we can arrive to a compensation case from the initial one .

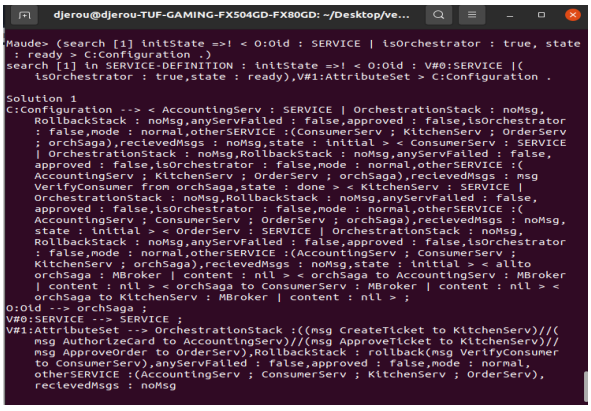


Fig. 4. Result of the execution search command for check if service orchestrate change to state ready .

We used this mark ' ! ' in examples for search to arrive final state . In the third example we checked if any service other than the orchestrator in the state rollbacked .

```
( search [1] initState = >! < O : Oid : SERVICE |
    isOrchestrator : false , state : rollbacked > C
    : Configuration .) --- search a Service not
    orchestrat in the state rollbacked .
```

Result can be displayed in figure 5.



Fig. 5. Result of the execution search command for check if any service not orchestrate in the state rollbacked .

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 08:52:45 UTC from IEEE Xplore. Restrictions apply.

We can use too all of command lines execution and checking in the same file of code source.

After that we used in or load commands in the terminal for reading the file.maude .

VI. Conclusion

The microservice based architecture exemplifies the idea of a distributed system. This architecture has a number of advantages. Managing the data exchanges between several services is one of the most challenging tasks in microservices where database per service enhances the concept of undependability of services, but it requires a distributed transaction management. This final one must guarantee the accuracy of a transaction in which each microservice is required to finish its own local transaction.

The microservice architecture can use two phase commit protocol but it is not suitable with microservice pattern. The saga pattern has emerged, alternative solution of two phase commit protocol. Using saga pattern, a set of transactions are produced that update microservices sequentially and locally. Throughout this paper, we have presented firstly the meaning of Maude language then we have explained the microservice architecture then we have focused on the Saga concept. Indeed, the paper has tackled illustrative example. We were particularly interested in the saga pattern's verification. We have proposed a specification for saga using maude language. We used this language in the orchestration-based to confirm this trend and to confirm the correctness of a given transaction across many micro-services. This formal language is accompanied with a set of commands allowing the verification of the specified pattern to check if a set of required properties are satisfied by the saga pattern. In future works, the current orchestration saga model will be enhanced with more details and properties according to the microservice architecture specification updates and will verify the distributed transaction of the second way of saga " choreography" using maude.

References

[1] G. Barthe, S. Blazy, B. Grégoire, R. Hutin, V. Laporte, D. Pichardie, and A. Trieu, "Formal verification of a constant-time preserving c compiler," *Proceedings of the ACM on Programming Languages*, vol. 4, no. POPL, pp. 1–30, 2020.

[2] A. Souri, N. J. Navimipour, and A. M. Rahmani, "Formal verification approaches and standards in the cloud computing: a comprehensive and systematic review," *Computer Standards & Interfaces*, vol. 58, pp. 1–22, 2018.

[3] H. E. Hafidi, Z. Hmidi, L. Kahloul, and S. Benharzallah, "Formal specification and verification of 5g authentication and key agreement protocol using mcrl2," in *2021 International Conference on Networking and Advanced Systems (ICNAS)*, pp. 1–6, 2021.

[4] M. Mirelli *et al.*, "A formal verification tool for lending pools," 2021.

[5] M. Amel and C. Allaoua, "Embedded system verification using formal model an approach based on the combined use of uml and maude language," *International Journal of Conceptual Structures and Smart Applications (IJCSSA)*, vol. 6, no. 2, pp. 42–58, 2018.

[6] S. Liu, P. C. Olveczky, K. Santhanam, Q. Wang, I. Gupta, and J. Meseguer, "A new distributed transaction protocol and its formal analysis in maude," tech. rep., 2018.

[7] A. Bakri, *Développement d'un système de vérification des systèmes embarqués*. PhD thesis, FACULTE DES MATHEMATIQUES ET DE L'INFORMATIQUE-UNIVERSITE MOHAMED BOUDIAF-M'SILA, 2017.

[8] M. Clavel, F. Durán, S. Eker, P. Lincoln, N. Martı-Oliet, J. Meseguer, and J. F. Quesada, "Maude: Specification and programming in rewriting logic," *Theoretical Computer Science*, vol. 285, no. 2, pp. 187–243, 2002.

[9] R. López Rueda, "Narrowing and unification in the maude programming language," 2021.

[10] J. Meseguer, "Conditional rewriting logic as a unified model of concurrency," *Theoretical computer science*, vol. 96, no. 1, pp. 73–155, 1992.

[11] J. Meseguer, "Twenty years of rewriting logic," *The Journal of Logic and Algebraic Programming*, vol. 81, no. 7, pp. 721–781, 2012. Rewriting Logic and its Applications.

[12] J. Meseguer, "Symbolic computation in maude: Some tapas," in *Logic-Based Program Synthesis and Transformation* (M. Fernández, ed.), (Cham), pp. 3–36, Springer International Publishing, 2021.

[13] K. Madani and S. Medjouri, "Réalisation d'un outil de génération automatique de spécification maude (nouvelle sémantique) des réseaux de petri.."

[14] W. Chama, R. Elmansouri, and A. Chaoui, "Using graph transformation and maude to simulate and verify uml models," in *2013 The International Conference on Technological Advances in Electrical, Electronics and Computer Engineering (TAEECE)*, pp. 459–464, IEEE, 2013.

[15] L. De Lauretis, "From monolithic architecture to microservices architecture," in *2019 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*, pp. 93–96, IEEE, 2019.

[16] M. Mazzara and B. Meyer, *Present and Ulterior Software Engineering*. 01 2018.

[17] C. Richardson, *Microservices Patterns: With examples in Java*. Manning, 2018.

[18] D. Manel and T. Okba, "Saga distributed transactions verification using maude." https://drive.google.com/drive/folders/1AEzcitVCfY9a-DrOC8-pfN5o87Zvw3Z5?usp=sharing.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 08:52:45 UTC from IEEE Xplore. Restrictions apply.